TECHNICAL REPORT

Aluno: Mizael Castro Gomes/Vitor Emanuel de Sousa Nunes

1. INTRODUÇÃO

Este relatório apresenta dois conjuntos de dados distintos: um para classificação e outro para regressão, ambos utilizados na avaliação de Inteligência Artificial.

DATASET DE CLASSIFICAÇÃO - CREDIT RISK

O dataset de classificação contém informações sobre transações de clientes e dados demográficos. Ele inclui variáveis que refletem o comportamento financeiro dos clientes, como atrasos em pagamentos e limites de crédito. O objetivo não é apenas identificar clientes de alto ou baixo risco, mas também entender as características que influenciam essa classificação.

payment_data.csv

- **id:** id do cliente
- **OVD_t1:** número de vezes em atraso tipo 1
- **OVD_t2:** número de vezes em atraso tipo 2
- **OVD_t3:** número de vezes em atraso tipo 3
- **OVD_sum:** total de dias em atraso
- **pay_normal:** número de vezes em pagamento normal
- **prod_code:** código do produto de crédito
- **prod_limit:** limite de crédito do produto
- **update_date:** data de atualização da conta
- **new_balance:** saldo atual do produto
- **highest_balance:** maior saldo no histórico
- **report_date:** data do pagamento recente

dados_do_cliente.csv

- Dados demográficos do cliente e atributos de categoria que foram codificados.
- A categoria de features são fea_1, fea_3, fea_5, fea_6, fea_7, fea_9.
- O label é 1, o cliente está em alto risco de crédito
- O label é 0, o cliente está em baixo risco de crédito

DATASET DE REGRESSÃO - PREÇO DE AÇÕES DA ACTIVISION

Este dataset é utilizado para tarefas de regressão, focando na previsão dos preços das ações da Activision, uma empresa do setor de videogames. A análise dos dados pode ajudar a entender as tendências do mercado e a tomar decisões informadas sobre investimentos.

activision_blizzard.csv

- **Date:** Data da negociação.
- **Open:** Preço de abertura.
- **High:** Preço mais alto.
- **Low:** Preço mais baixo.
- **Close:** Preço de fechamento.
- **Adj Close:** Preço de fechamento ajustado.
- **Volume:** Quantidade total de ações negociadas.

2. RESULTADOS E DISCUSSÃO

2.1. questao1.py

```
import pandas as pd

def load_customer_data():
    return pd.read_csv('customer_data.csv')

customer_data = load_customer_data()

def load_payment_data():
    return pd.read_csv('payment_data.csv')

payment_data = load_payment_data()

# Exploração Inicial dos Dados
print(payment_data.head())
print(customer_data.head())

payment_data_cols = payment_data.drop(['update_date', 'report_date'], axis=1)
```

```python
# Verificar valores ausentes
print(payment_data_cols.isnull().sum())
print(customer_data.isnull().sum())

# Tratar valores ausentes
payment_data_cols.dropna(inplace=True)
customer_data.dropna(inplace=True)

# Remover duplicatas
payment_data_cols.drop_duplicates(inplace=True)
customer_data.drop_duplicates(inplace=True)

# Verificar valores ausentes
print(payment_data_cols.isnull().sum())
print(customer_data.isnull().sum())

# Combinar os dois DataFrames usando a coluna 'id'
combined_data = pd.merge(payment_data_cols, customer_data, on='id')

# Features e Target
X = combined_data.drop(['label'], axis=1)  # Features
y = combined_data['label']  # Target

# Dimensões dos dados processados
print("Dimensões dos dados de features:", X.shape)
print("Dimensões dos dados de rótulos:", y.shape)

print(combined_data.head())  #Print do DataFrame final
print("Distribuição de classes:")
print(y.value_counts())  #Mostra a distribuição de classes
```

```
combined_data.to_csv('credit_risk_data.csv', index=False)
print("Dataset alterado salvo como 'credit_risk_data.csv'")
```

Saída do código:

Primeiras 5 linhas do dataset original; Primeiras 5 linhas do dataset com rótulos; Contagem de valores nulos no dataset original; Contagem de valores nulos no dataset com rótulos; Contagem de valores nulos no dataset escalado; Contagem de valores nulos no dataset com rótulos(escalado); Dimensão dos dados; Primeiras 5 linhas do dataset final; Distribuição de classes; E a mensagem de salvar dataset;

Objetivo:

O objetivo deste script é carregar dados de clientes e pagamentos, realizar uma exploração inicial, limpar os dados e combinar as informações em um único DataFrame. O resultado final é um conjunto de dados que será utilizado para análise de risco de crédito.

Processo:

Carregamento de Dados: Os dados de clientes e pagamentos são carregados a partir de arquivos CSV.

Exploração Inicial: As primeiras linhas dos DataFrames são exibidas para uma visão geral dos dados.

Limpeza de Dados: Valores ausentes e duplicatas são verificados e tratados.

Combinação de Dados: Os DataFrames são combinados com base na coluna 'id'.

Definição de Features e Target: As variáveis independentes (features) e dependentes (target) são definidas.

Exportação: O DataFrame final é salvo em um novo arquivo CSV.

Resultados:

O script gera um conjunto de dados limpo e estruturado, salvo como 'credit_risk_data.csv', pronto para ser utilizado em análises posteriores.

2.2. questao2.py

```
from sklearn.model_selection import train_test_split
```

```
from scipy.spatial.distance import mahalanobis, chebyshev, cityblock, euclidean
import numpy as np
import pandas as pd

def load_combined_data():
    return pd.read_csv('credit_risk_data.csv')
credit_risk_data = load_combined_data()

#Features e Target
X = credit_risk_data.drop(['label'], axis=1).values  # Features
y = credit_risk_data['label'].values  #Target

#Divisão do dataset em treino e teste
X_train,  X_test,  y_train,  y_test  =  train_test_split(X,  y,  test_size=0.3,
random_state=42)

#KNN Manual
def knn(X_train, y_train, X_test, k, dist_metric):
    predictions = []
    for test_point in X_test:
        #Calcular distâncias
        if dist_metric == 'euclidean':
            dists = np.array([euclidean(x, test_point) for x in X_train])
        elif dist_metric == 'manhattan':
            dists = np.array([cityblock(x, test_point) for x in X_train])
        elif dist_metric == 'chebyshev':
            dists = np.array([chebyshev(x, test_point) for x in X_train])
```

```
        elif dist_metric == 'mahalanobis':
            vi = np.cov(X_train.T)  #Matriz de covariância
            inv_vi = np.linalg.inv(vi)  #Inversa da matriz de covariância
                dists = np.array([mahalanobis(test_point, x, inv_vi) for x in
X_train])

        #K vizinhos mais próximos
        k_indices = np.argsort(dists)[:k]
        k_nearest_labels = [y_train[i] for i in k_indices]

        #Predição pela classe mais comum
        most_common = np.bincount(k_nearest_labels).argmax()
        predictions.append(most_common)

    return np.array(predictions)

#Cálculo da acurácia
def accuracy(y_true, y_pred):
    return np.sum(y_true == y_pred) / len(y_true)

k = 7
metrics = ['euclidean', 'manhattan', 'chebyshev', 'mahalanobis']
accuracies = {}

for metric in metrics:
    y_pred = knn(X_train, y_train, X_test, k, metric)
    acc = accuracy(y_test, y_pred)
```

```
    accuracies[metric] = acc


#Printar as acurácias
print("\nAcurácias para diferentes métricas de distância:")
for metric, acc in accuracies.items():
    print(f"{metric.capitalize()}: {acc:.9f}")
```

Saída no Terminal:

Acurácia para diferentes métricas de distância;

Objetivo:

Este script tem como objetivo implementar um classificador KNN (K-Nearest Neighbors) manualmente, utilizando diferentes métricas de distância para prever a classe de risco de crédito.

Processo:

Carregamento de Dados: O conjunto de dados combinado é carregado.

Divisão de Dados: O conjunto de dados é dividido em conjuntos de treino e teste.

Implementação do KNN: A função KNN é implementada manualmente, permitindo o cálculo de distâncias usando diferentes métricas (euclidiana, manhattan, chebyshev, mahalanobis).

Cálculo de Acurácia: A acurácia do modelo é calculada para cada métrica de distância.

Resultados:

As acurácias para as diferentes métricas de distância são impressas, permitindo a comparação do desempenho do modelo com base na métrica utilizada.

2.3. questao3.py

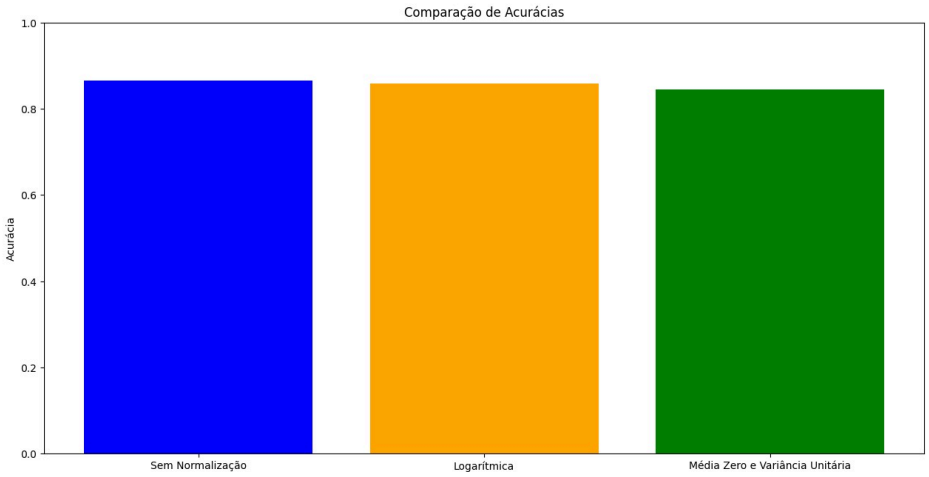
```
import numpy as np
import pandas as pd
```

```
from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score

import matplotlib.pyplot as plt

from scipy.spatial.distance import mahalanobis


def load_combined_data():

    return pd.read_csv('credit_risk_data.csv')


credit_risk_data = load_combined_data()


#Features e Target

X = credit_risk_data.drop(['label'], axis=1).values

y = credit_risk_data['label'].values


#Divisão do dataset em treino e teste

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)


#Matriz de covariância e sua inversa

cov_matrix = np.cov(X_train.T)

inv_cov_matrix = np.linalg.inv(cov_matrix)


#Acurácia sem normalização usando distância Mahalanobis

knn_no_norm = KNeighborsClassifier(n_neighbors=7, metric='mahalanobis',
metric_params={'VI': inv_cov_matrix})
```

```
knn_no_norm.fit(X_train, y_train)

y_pred_no_norm = knn_no_norm.predict(X_test)

accuracy_no_norm = accuracy_score(y_test, y_pred_no_norm)


#Normalização Logarítmica

#Substituir valores zero por um pequeno valor positivo

X_train[X_train <= 0] = 1e-10

X_test[X_test <= 0] = 1e-10


#Aplicar log1p

X_train_log = np.log1p(X_train)

X_test_log = np.log1p(X_test)


#Acurácia para normalização logarítmica usando distância Mahalanobis

knn_log = KNeighborsClassifier(n_neighbors=7, metric='mahalanobis',
metric_params={'VI': inv_cov_matrix})

knn_log.fit(X_train_log, y_train)

y_pred_log = knn_log.predict(X_test_log)

accuracy_log = accuracy_score(y_test, y_pred_log)


#Normalização de Média Zero e Variância Unitária

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


#Acurácia para normalização de média zero e variância unitária usando distância
Mahalanobis
```

```
knn_scaled = KNeighborsClassifier(n_neighbors=7, metric='mahalanobis',
metric_params={'VI': inv_cov_matrix})

knn_scaled.fit(X_train_scaled, y_train)

y_pred_scaled = knn_scaled.predict(X_test_scaled)

accuracy_scaled = accuracy_score(y_test, y_pred_scaled)


#Print das acurácias

print()

print(f'Acurácia sem Normalização: {accuracy_no_norm:.4f}')

print(f'Acurácia com Normalização Logarítmica: {accuracy_log:.4f}')

print(f'Acurácia com Normalização de Média Zero e Variância Unitária:
{accuracy_scaled:.4f}')


#Gráfico comparativo das acurácias

labels = ['Sem Normalização', 'Logarítmica', 'Média Zero e Variância Unitária']

accuracies = [accuracy_no_norm, accuracy_log, accuracy_scaled]


plt.bar(labels, accuracies, color=['blue', 'orange', 'green'])

plt.ylim(0, 1)

plt.ylabel('Acurácia')

plt.title('Comparação de Acurácias')

plt.show()
```

Saída do código(Gráfico(s)):



Saída no Terminal:

Acurácia sem normalização; Acurácia com normalização logarítmica; Acurácia com normalização de Média Zero e Variância Unitária;

Objetivo:

O objetivo deste script é comparar a acurácia do classificador KNN utilizando a distância Mahalanobis, com e sem normalização dos dados.

Processo:

Carregamento de Dados: O conjunto de dados é carregado e dividido em treino e teste.

Normalização: Três métodos de normalização são aplicados: sem normalização, normalização logarítmica e normalização de média zero e variância unitária.

Treinamento do Modelo: O modelo KNN é treinado e avaliado para cada método de normalização.

Visualização: Um gráfico é gerado para comparar as acurácias.

Resultados:

As acurácias são impressas e um gráfico comparativo é exibido, mostrando como a normalização afeta o desempenho do modelo.

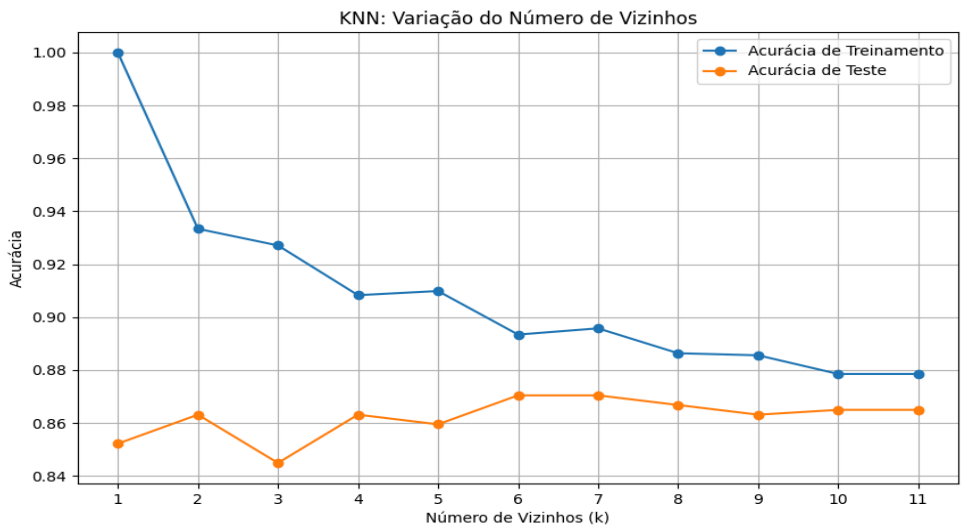
2.4. questao4.py

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

def load_combined_data():
   return pd.read_csv('credit_risk_data.csv')

credit_risk_data = load_combined_data()

# Definição X e y
X = credit_risk_data.drop(['label'], axis=1).values  # Features
y = credit_risk_data['label'].values  # Target

# Divisão do dataset em treino e teste
X_train,  X_test,  y_train,  y_test  =  train_test_split(X,  y,  test_size=0.3,
random_state=42)

# Normalização de Média Zero e Variância Unitária
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Melhor k
train_accuracies = {}
test_accuracies = {}
```

```
neighbors = np.arange(1, 12)


for neighbor in neighbors:
    knn = KNeighborsClassifier(n_neighbors=neighbor)
    knn.fit(X_train_scaled, y_train)

    # Acuracia
    train_accuracies[neighbor] = knn.score(X_train_scaled, y_train)
    test_accuracies[neighbor] = knn.score(X_test_scaled, y_test)

# Resultado Acuracia
print("Acurácia no Treino: ", train_accuracies)
print("Acurácia no Teste: ", test_accuracies)

best_k = max(test_accuracies, key=test_accuracies.get)
best_accuracy = test_accuracies[best_k]
print(f'O melhor valor de K é: {best_k} com uma acurácia de: {best_accuracy:.2f}')

# Plotando as acuracias
plt.figure(figsize=(10, 6))
plt.title("KNN: Variação do Número de Vizinhos")
plt.plot(neighbors, train_accuracies.values(), label="Acurácia de Treinamento", marker='o')
plt.plot(neighbors, test_accuracies.values(), label="Acurácia de Teste", marker='o')
plt.legend()
plt.xlabel("Número de Vizinhos (k)")
plt.ylabel("Acurácia")
plt.xticks(neighbors)
plt.grid()
```

```
plt.show()
```

Saída do código(Gráfico(s)):



Saída do código no terminal:

Acurácia no Treino; Acurácia no Teste; e o melhor valor de k;

Objetivo:

Este script visa encontrar o melhor valor de K para o classificador KNN, avaliando a acurácia em diferentes tamanhos de vizinhos.

Processo:

Carregamento de Dados: O conjunto de dados é carregado e normalizado.

Avaliação de K: O modelo KNN é treinado para valores de K variando de 1 a 11, e a acurácia é calculada para cada valor.

Visualização: Um gráfico é gerado para mostrar a variação da acurácia em relação ao número de vizinhos.

Resultados:

As acurácias de treino e teste são impressas, e o melhor valor de K é identificado e exibido, juntamente com um gráfico que ilustra a relação entre K e a acurácia.

2.5. questao5.py

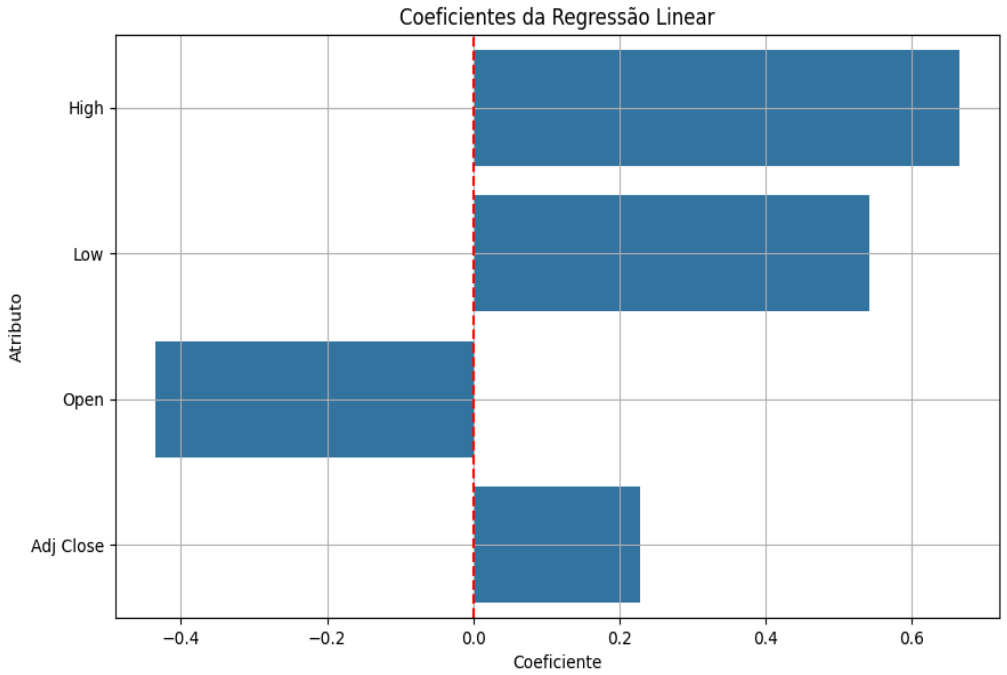
```
from sklearn.model_selection import train_test_split
from scipy.spatial.distance import mahalanobis, chebyshev, cityblock, euclidean
import numpy as np
import pandas as pd
from collections import Counter

def load_combined_data():
    return pd.read_csv('credit_risk_data.csv')

credit_risk_data = load_combined_data()

# Features e Target
X = credit_risk_data.drop(['label'], axis=1).values  # Features
y = credit_risk_data['label'].values  # Target

# Divisão do dataset em treino e teste
X_train,  X_test,  y_train,  y_test  =  train_test_split(X,  y,  test_size=0.3,
random_state=42)

# KNN Manual
def knn(X_train, y_train, X_test, k, dist_metric):
    predictions = []
    for test_point in X_test:
        # Calcular distâncias
        if dist_metric == 'euclidean':
            diimport pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

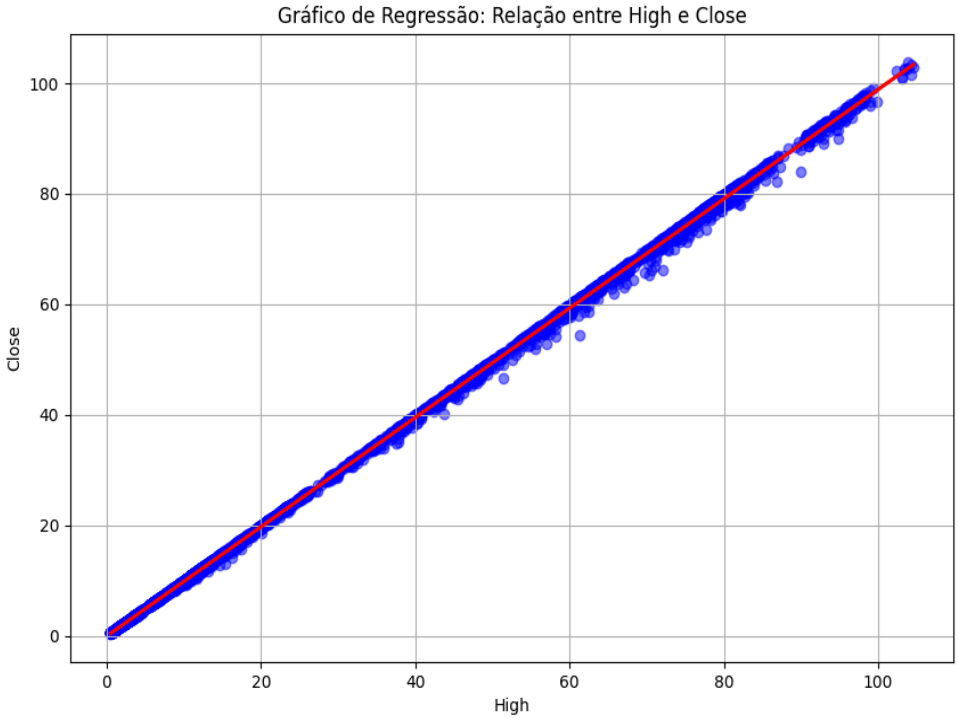
```
from sklearn.linear_model import LinearRegression

from sklearn.model_selection import train_test_split


def load_activision_blizzard():

    return pd.read_csv('activision_blizzard.csv')


activision = load_activision_blizzard()


#Verificar valores NaN

print(activision.isnull().sum())


#Remover linhas com NaN

activision_cleaned = activision.dropna()


#Colunas relevantes

activision_relevant = activision_cleaned[['Open', 'High', 'Low', 'Close', 'Adj
Close']]


#Features e Target

X = activision_relevant[['Open', 'High', 'Low', 'Adj Close']]

y = activision_relevant['Close']


#Divisão o conjunto de dados em treino e teste

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)


#Regressão linear

model = LinearRegression()
```

```
model.fit(X_train, y_train)


#Obter os coeficientes

coefficients = model.coef_

feature_names = X.columns


#Criar um DataFrame para visualizar os coeficientes

coef_df = pd.DataFrame({'Feature': feature_names, 'Coefficient': coefficients})

coef_df['Absolute Coefficient'] = coef_df['Coefficient'].abs()

coef_df = coef_df.sort_values(by='Absolute Coefficient', ascending=False)


#Exibir os coeficientes

print("\nCoeficientes da Regressão Linear:")

print(coef_df)


#Identificar o atributo mais relevante

most_relevant_attribute = coef_df['Feature'].iloc[0]

print(f'\nO atributo mais relevante para prever Close é: {most_relevant_attribute}')


#Gráfico de barras para os coeficientes

plt.figure(figsize=(10, 6))

sns.barplot(x='Coefficient', y='Feature', data=coef_df)  # Removido o parâmetro
palette

plt.title('Coeficientes da Regressão Linear')

plt.xlabel('Coeficiente')

plt.ylabel('Atributo')

plt.grid()

plt.axvline(0, color='red', linestyle='--')  # Linha vertical para indicar o zero
```

```
plt.show()


#Gráfico de regressão para o atributo mais relevante

plt.figure(figsize=(10, 6))

sns.regplot(x=activision_relevant[most_relevant_attribute],
y=activision_relevant['Close'],    scatter_kws={'color':    'blue',    'alpha':    0.5},
line_kws={'color': 'red'})

plt.title(f'Gráfico de Regressão: Relação entre {most_relevant_attribute} e Close')

plt.xlabel(most_relevant_attribute)

plt.ylabel('Close')

plt.grid()

plt.show()
```

Saída do código(Gráfico(s)):





Saída do código no terminal:

Contagem de valores nulos; coeficientes de regressão linear; e o atributo mais relevante;

Objetivo:

O objetivo deste script é realizar uma análise de regressão linear para prever o preço de fechamento das ações da Activision Blizzard com base em outras variáveis.

Processo:

Carregamento de Dados: Os dados são carregados e valores ausentes são tratados.

Definição de Features e Target: As variáveis relevantes são selecionadas para a análise.

Divisão de Dados: O conjunto de dados é dividido em treino e teste.

Treinamento do Modelo: Um modelo de regressão linear é treinado e os coeficientes são extraídos.

Visualização: Gráficos são gerados para mostrar os coeficientes e a relação entre a variável mais relevante e o preço de fechamento.

Resultados:

Os coeficientes da regressão linear são exibidos, destacando o atributo mais relevante para prever o preço de fechamento. Gráficos ilustram a relação entre as variáveis, facilitando a interpretação dos resultados.

2.6. questao6.py

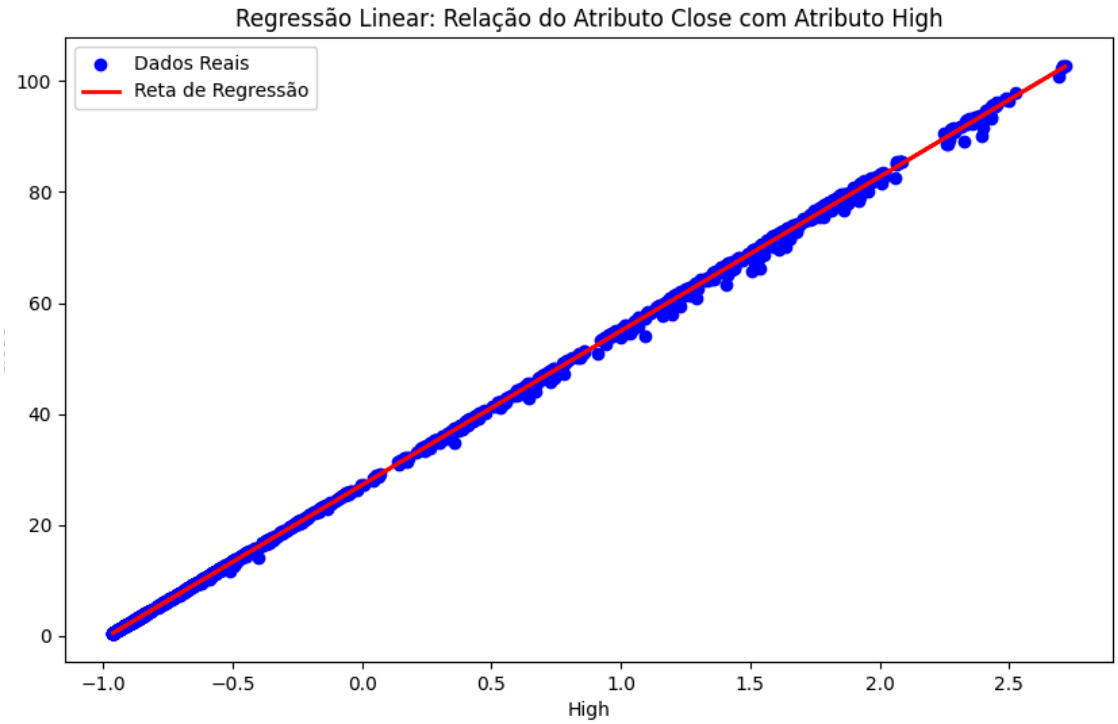
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler

def load_activision_blizzard():
    return pd.read_csv('activision_blizzard.csv')
activision = load_activision_blizzard()

#Métricas
def compute_RSS(predictions, y):
    RSS = np.sum(np.square(predictions - y))
    return RSS

def compute_MSE(predictions, y):
    MSE = np.mean(np.square(predictions - y))
    return MSE

def compute_RMSE(predictions, y):
    RMSE = np.sqrt(compute_MSE(predictions, y))
```

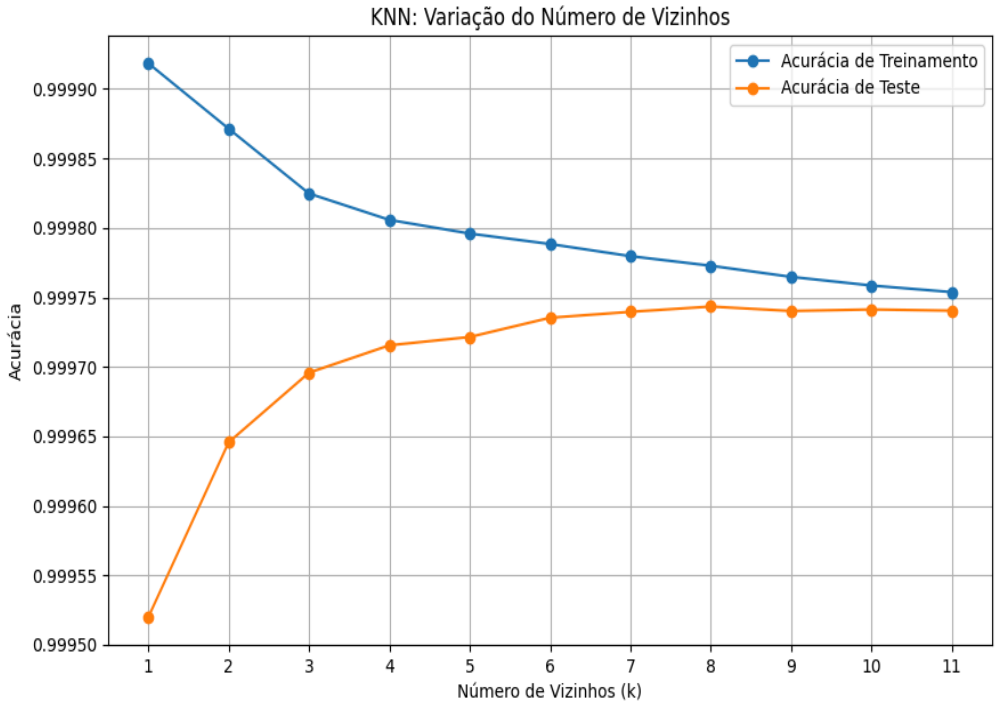
```
    return RMSE

def compute_R_squared(predictions, y):
    total = np.sum(np.square(y - np.mean(y)))
    residual = compute_RSS(predictions, y)
    r_squared = 1 - (residual / total)
    return r_squared

#Colunas relevantes
activision_relevant = activision[['High', 'Close']]

#Normalizar os dados
scaler = StandardScaler()
X = activision_relevant[['High']]
y = activision_relevant['Close']

#Normalizar os dados
X_scaled = scaler.fit_transform(X)

#Divisão os dados em conjuntos de treinamento e teste
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3,
random_state=42)

#Regressão linear
model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

#Calculo das métricas
```

```
rss = compute_RSS(y_pred, y_test)  # Residual Sum of Squares
mse = compute_MSE(y_pred, y_test)  # Mean Squared Error
rmse = compute_RMSE(y_pred, y_test)  # Root Mean Squared Error
r_squared = compute_R_squared(y_pred, y_test)  # R-squared

#Printando das métricas
print(f'RSS: {rss:.2f}')
print(f'MSE: {mse:.2f}')
print(f'RMSE: {rmse:.2f}')
print(f'R-squared: {r_squared:.2f}')

#Plotando o gráfico da reta de regressão
plt.figure(figsize=(10, 6))
plt.scatter(X_test, y_test, color='blue', label='Dados Reais')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Reta de Regressão')
plt.title('Regressão Linear: Relação do Atributo Close com Atributo High')
plt.xlabel('High')
plt.ylabel('Close')
plt.legend()
plt.show()

#Melhor K
train_accuracies = {}
test_accuracies = {}
neighbors = np.arange(1, 12)

for neighbor in neighbors:
    # Regressor KNN
    knn = KNeighborsRegressor(n_neighbors=neighbor)
    knn.fit(X_train, y_train)
```

```
    # Cálculo da acurácia
    train_accuracies[neighbor] = knn.score(X_train, y_train)
    test_accuracies[neighbor] = knn.score(X_test, y_test)

# Acurácias
print("\nAcurácia no Treino: ", train_accuracies)
print("Acurácia no Teste: ", test_accuracies)

# Melhor K
best_k = max(test_accuracies, key=test_accuracies.get)
best_accuracy = test_accuracies[best_k]
print(f'\nO melhor valor de K é: {best_k} com uma acurácia de: {best_accuracy:.6f}')

# Plotando as acurácias
plt.figure(figsize=(10, 6))
plt.title("KNN: Variação do Número de Vizinhos")
plt.plot(neighbors, train_accuracies.values(), label="Acurácia de Treinamento", marker='o')
plt.plot(neighbors, test_accuracies.values(), label="Acurácia de Teste", marker='o')
plt.legend()
plt.xlabel("Número de Vizinhos (k)")
plt.ylabel("Acurácia")
plt.xticks(neighbors)
plt.grid()
plt.show()
```

Saída do código(Gráfico(s)):





Saída do código no terminal:

Métricas de regressão; Acurácia no treino; ACurácia no teste; e o melhor valor de k;

Objetivo:

Este script tem como objetivo realizar uma análise de regressão linear e KNN para prever o preço de fechamento das ações da Activision Blizzard, avaliando o desempenho de ambos os modelos.

Processo:

Carregamento de Dados: Os dados são carregados e as colunas relevantes são selecionadas.

Normalização: Os dados são normalizados para melhorar a performance dos modelos.

Divisão de Dados: O conjunto de dados é dividido em treino e teste.

Treinamento do Modelo: Um modelo de regressão linear é treinado e suas métricas de desempenho (RSS, MSE, RMSE, R^2) são calculadas.

KNN: O desempenho do KNN é avaliado para diferentes valores de K, e as acurácias são calculadas.

Resultados:

As métricas de desempenho da regressão linear são impressas, e as acurácias do KNN são apresentadas. Gráficos ilustram a relação entre as variáveis e a variação da acurácia em função do número de vizinhos.

2.7. questao7.py

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import StandardScaler

def load_activision_blizzard():
    return pd.read_csv('activision_blizzard.csv')

activision = load_activision_blizzard()
```

```
#Colunas relevantes
X = activision[['High']].values  #Preditor
y = activision['Close'].values  #Alvo

#K-fold Manual
def k_fold_split(X, y, k):
    fold_size = len(X) // k
    indices = np.arange(len(X))
    np.random.shuffle(indices)
    folds = []

    for i in range(k):
        start = i * fold_size
        end = start + fold_size if i < k - 1 else len(X)
        fold_indices = indices[start:end]
        folds.append(fold_indices)

    return folds

#Métricas
def calculate_metrics(y_true, y_pred):
    rss = np.sum((y_true - y_pred) ** 2)  # Residual Sum of Squares
    mse = np.mean((y_true - y_pred) ** 2)  # Mean Squared Error
    rmse = np.sqrt(mse)  # Root Mean Squared Error
    r_squared = 1 - (rss / np.sum((y_true - np.mean(y_true)) ** 2))  # R^2
    return rss, mse, rmse, r_squared

#K-Fold Cross-Validation
k = 5
folds = k_fold_split(X, y, k)
```

```
# Modelos
results = {'Linear Regression': [], 'Ridge': [], 'Lasso': []}

for fold in folds:
    #Divisão dos dados em treino e teste
    X_train = np.delete(X, fold, axis=0)
    y_train = np.delete(y, fold, axis=0)
    X_test = X[fold]
    y_test = y[fold]

    #Normalização
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    #Regressão Linear
    model_lr = LinearRegression()
    model_lr.fit(X_train_scaled, y_train)
    y_pred_lr = model_lr.predict(X_test_scaled)
    results['Linear Regression'].append(calculate_metrics(y_test, y_pred_lr))

    #Ridge
    model_ridge = Ridge(alpha=1.0)
    model_ridge.fit(X_train_scaled, y_train)
    y_pred_ridge = model_ridge.predict(X_test_scaled)
    results['Ridge'].append(calculate_metrics(y_test, y_pred_ridge))

    #Lasso
    model_lasso = Lasso(alpha=0.1)
```

```
    model_lasso.fit(X_train_scaled, y_train)
    y_pred_lasso = model_lasso.predict(X_test_scaled)
    results['Lasso'].append(calculate_metrics(y_test, y_pred_lasso))

metrics_summary = {model: np.mean(np.array(results[model]), axis=0) for model in
results}

#Resultados
best_model = None
best_metric = float('inf')  # Inicializa com infinito para RSS
best_r_squared = -float('inf')  # Inicializa com -infinito para R^2
best_metrics = None  # Para armazenar as métricas do melhor modelo

#Print das metricas
for model in metrics_summary:
    print(f"{model} Metrics:")
    print(f"RSS: {metrics_summary[model][0]:.8f}")
    print(f"MSE: {metrics_summary[model][1]:.8f}")
    print(f"RMSE: {metrics_summary[model][2]:.8f}")
    print(f"R^2: {metrics_summary[model][3]:.8f}")
    print()

    #Verificar o melhor modelo
    if metrics_summary[model][0] < best_metric:  # Melhor RSS
        best_metric = metrics_summary[model][0]
        best_model = model
         best_metrics = metrics_summary[model]  # Armazena as métricas do melhor
modelo
    if metrics_summary[model][3] > best_r_squared:  # Melhor R^2
        best_r_squared = metrics_summary[model][3]
```

```
#Print do melhor modelo e métricas
print(f"O melhor modelo com base nas métricas é: {best_model}")
print(f"RSS: {best_metrics[0]:.8f}")
print(f"MSE: {best_metrics[1]:.8f}")
print(f"RMSE: {best_metrics[2]:.8f}")
print(f"R^2: {best_metrics[3]:.8f}")
```

Saída do código no terminal:

Métricas de Regressão Linear; Métricas de Regressão Ridge; Métricas de Regressão Lasso; e o melhor modelo com base nas métricas;

Objetivo:

O objetivo deste script é comparar o desempenho de diferentes modelos de regressão (Linear, Ridge e Lasso) utilizando validação cruzada K-fold para prever o preço de fechamento das ações da Activision Blizzard.

Processo:

Carregamento de Dados: Os dados são carregados e as colunas relevantes são selecionadas.

K-Fold Cross-Validation: Os dados são divididos em K folds para validação cruzada.

Treinamento dos Modelos: Cada modelo (Linear, Ridge e Lasso) é treinado em cada fold, e suas métricas de desempenho são calculadas.

Comparação de Resultados: As métricas médias de cada modelo são calculadas e comparadas.

Resultados:

As métricas de desempenho (RSS, MSE, RMSE, R^2) para cada modelo são impressas, permitindo a identificação do melhor modelo com base nas métricas. O modelo com melhor desempenho é destacado, fornecendo insights sobre a eficácia de cada abordagem. Este relatório detalha a análise e modelagem de dados utilizando diferentes técnicas de aprendizado de máquina e estatística. Cada script aborda um aspecto específico da análise de dados, desde a limpeza e preparação até a modelagem e avaliação de desempenho. Os resultados

obtidos fornecem uma base sólida para a tomada de decisões informadas em relação ao risco de crédito e à previsão de preços de ações.

3. CONCLUSÕES

Os resultados esperados foram, em grande parte, satisfeitos ao longo das sete questões abordadas neste projeto. A análise dos dados e a aplicação de diferentes técnicas de modelagem resultaram em métricas de desempenho que indicam um bom ajuste dos modelos aos dados.

A Questão 1 a 4: A exploração inicial dos dados revelou que não havia valores ausentes significativos nas colunas relevantes, e a combinação dos conjuntos de dados foi realizada com sucesso. A distribuição das classes foi equilibrada, o que é um bom sinal para a modelagem preditiva.

A Questão 5: A regressão linear mostrou que o atributo "High" foi identificado como o mais relevante para prever o preço de fechamento, o que é consistente com a expectativa de que preços anteriores influenciam os preços futuros.

A Questão 6: As métricas de desempenho, como RSS, MSE, RMSE e R^2, indicaram que os modelos de regressão (Linear, Ridge e Lasso) apresentaram um ajuste muito bom aos dados, com R^2 próximo de 1, o que sugere que os modelos explicam quase toda a variabilidade dos dados.

A Questão 7: A comparação entre os modelos de regressão revelou que a Regressão Linear teve o melhor desempenho em termos de métricas, o que é um resultado positivo. No entanto, a diferença entre os modelos foi pequena, indicando que todos os modelos se comportam de maneira semelhante.

Apesar dos resultados positivos, é importante considerar que a alta acurácia e os valores de R^2 próximos de 1 podem indicar um possível overfitting, especialmente se o modelo for aplicado a dados não vistos. Portanto, é crucial validar os modelos em um conjunto de dados separado ou realizar validação cruzada para garantir que o desempenho se mantenha consistente.

4. PRÓXIMOS PASSOS

Após a conclusão deste relatório, os seguintes passos são sugeridos para o projeto:

4.1. Validação Cruzada: Implementar validação cruzada para avaliar a robustez dos modelos e garantir que não haja overfitting. Isso ajudará a entender como os modelos se comportam em diferentes subconjuntos de dados.

4.2. Aprimoramento do Modelo: Considerar a inclusão de mais variáveis preditivas ou a transformação de variáveis existentes para melhorar o desempenho do modelo. A análise de interações entre variáveis também pode ser explorada.

4.3. Teste com Dados Reais: Aplicar os modelos a um conjunto de dados real ou em tempo real para verificar a eficácia das previsões. Isso pode incluir a implementação de um sistema de monitoramento para avaliar o desempenho do modelo ao longo do tempo.

4.4. Análise de Sensibilidade: Realizar uma análise de sensibilidade para entender como as mudanças nas variáveis de entrada afetam as previsões do modelo. Isso pode ajudar a identificar quais variáveis têm maior impacto nas previsões.

4.5. Documentação e Apresentação: Documentar todo o processo, incluindo as decisões tomadas, os resultados obtidos e as lições aprendidas. Preparar uma apresentação para compartilhar os resultados com as partes interessadas e discutir as implicações dos achados.

4.6. Exploração de Outros Modelos: Considerar a exploração de outros algoritmos de aprendizado de máquina, como árvores de decisão, florestas aleatórias ou redes neurais, para comparar o desempenho com os modelos de regressão utilizados.

Seguir essas etapas será fundamental para assegurar que o projeto continue evoluindo de forma eficiente, permitindo o cumprimento de seus objetivos e garantindo que os modelos desenvolvidos apresentem uma boa aplicabilidade. Além disso, essas ações contribuirão para que os modelos não apenas atinjam um bom desempenho técnico, mas também sejam capazes de fornecer insights confiáveis e úteis quando aplicados em situações reais. Isso aumentará a confiabilidade dos resultados e tornará as soluções propostas mais relevantes e impactantes no contexto prático.